IEEESTEC – 19th Student Projects Conference, Niš, 2026.
https://doi.org/organizer will add number later
Original scientific paper

Fiducial: Keeping Multidisciplinary Products Built with AI Agents Consistent

Aleksa Zdravković
12th grade high school student
ETŠ "Nikola Tesla" Niš
Niš, Serbia
aleksazdravkovic@proton.me
ORCID: 0009-0004-1535-1839

***Abstract*—This paper presents Fiducial, an open-source platform for building products that span software and hardware with AI coding agents. Agents now write each part of a product quickly, but the facts the parts share are still copied between them and drift apart. Fiducial declares every fact once, generates every artifact from it – code, translations, circuit board and enclosure – and checks every change, so the parts stay consistent with each other; whether a declared fact is itself correct remains for the person to judge. On three sets of 20 realistic mistakes, each written after the checks it tests, the checks caught 15, 16 and 14 on first run, where the same projects' ordinary CI caught none. Every miss fell into one of five classes, each now closed by one mechanism.**

Keywords—AI agents, single source of truth, code generation, consistency checking, hardware–software co-design, continuous integration

I. Introduction

Building a product used to take a team: someone for the circuit board, someone for the firmware, others for the app and the enclosure. AI coding agents such as Claude Code [1] and Codex [2] are changing that: a firmware driver, a web page or a script that models a case can be written in minutes, so one person can now attempt a whole product.

Building such products – first Outreach [3], an off-grid messaging system of radios, gateways, an app and printed cases, and then the platform described here – has shown where the difficulty has moved. Writing each part is no longer the slow step; keeping the parts in agreement is. In Outreach, the firmware and the app disagreed on the layout of a message, and a printed case did not fit its board. Neither part was wrong on its own: a shared fact had been copied by hand, one copy changed, and the mistake appeared only when the parts met (Fig. 1a).

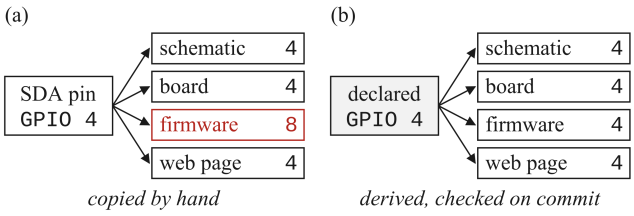


Fig. 1. One fact used in four places: (a) copied by hand, one copy changes and nothing notices; (b) declared once, every copy is generated from it and checked.

Agents speed this problem up rather than remove it. An agent sees only the codebase it works in, and its output is usually right but never guaranteed; more code, written faster in more disconnected places, gives an inconsistent product sooner.

What is missing is one shared and checked context for every discipline: a single place where a product's facts live, from which every part is produced, and which refuses a change that makes the parts disagree (Fig. 1b). This paper presents Fiducial [4], an open-source platform built to provide it, compares it with existing approaches and with ordinary CI, and tests its checks on mistakes they were not built for.

II. Existing Approaches

Several kinds of tool address parts of this problem; each keeps one kind of fact, or one pair of disciplines, in agreement, but not the product as a whole (Table I).

Model-driven and model-based systems engineering keep a design in one central model and generate or check other views from it [5], [6], in languages such as SysML [7], with methods for managing inconsistencies between views [8], [9]. They are the closest in spirit, but the model lives in a specialised graphical tool, code is still written by hand beside it, and an agent cannot easily work in it. Engineering suites instead link two tools, such as a circuit board and its mechanical model [10]; the firmware, the app and the translations stay outside.

Design-as-code tools write hardware as text – a circuit in atopile [11], a case in build123d [12] – which makes it writable by an agent, but each covers one discipline: Fiducial's first design planned to use atopile and found that the case, the layout and the cost would still copy the board's facts. Interface languages such as Protocol Buffers [13] declare a message once for both sides, which removes the drift met in Outreach, but only for messages.

TABLE I. EXISTING APPROACHES AND WHERE THEY STOP

Approach	Keeps in agreement	Stops at
Model-based engineering	views of one model	graphical tool; code by hand
ECAD–MCAD links	board and case	two disciplines, two tools
Design as code	one discipline	facts shared with others
Interface languages	messages, both sides	everything else
Build systems, CI	outputs with files	two files that disagree
Agent context files	nothing; they advise	not enforced
Fiducial	every declared fact	facts not yet declared

Build systems such as Bazel [14] and Nix [15] rebuild exactly what changed, identifying results by fingerprints of their inputs. They make sure outputs follow from files, not that files agree with each other: the same fact typed into two files builds without complaint, in them and in a CI pipeline built on them.

Coding agents follow context files such as AGENTS.md [16], which describe a repository's rules but do not enforce them, and the code they write is often plausible but wrong. Stricter tests find errors in programs that benchmark tests [17] had passed [18], about 40 % of the programs in one security study were vulnerable [19], and most real repository issues remain unsolved [20]. Models also cannot reliably correct their own reasoning without an outside signal [21]. A known answer is to pair a generator that can be wrong with a checker that cannot [22], but no existing checker spans a whole product.

Fiducial keeps what works in each – text that agents can write, outputs tied to inputs by fingerprints, established engines for hard problems, and checks that decide – and changes the unit. It tracks facts rather than files or tools, across every discipline of one product, so any fact – a message, a pin, a colour, a size – has exactly one home.

III. THE APPROACH

Fiducial treats a product as facts and artifacts. Facts are the decisions that define it: which pin a sensor uses, what a button says in each language, the size of the board, the layout of a message. Artifacts are everything built from them: code, schematics, boards, translations, enclosure models. Drift is when an artifact stops agreeing with a fact, or with another artifact that uses it (Fig. 1a). Three rules keep drift out (Fig. 1b).

Declare once. Every fact is written in exactly one place, a declaration: a typed text file in the repository, with a structure a program can check. A fact that exists once has no second copy to disagree with.

Derive everything else. Every artifact that depends on a fact is generated from its declaration by one command and never edited by hand. Generation is deterministic, so the product is reproducible and any difference is meaningful.

Check every change. Before a change is accepted, checks confirm that every artifact still matches what its declarations produce and that the declarations are valid; a failure names the declaration and the file at fault.

The rules check agreement, not truth. A fact declared wrongly – the wrong sensor address, a resistor of the wrong value – is copied faithfully into every artifact, and the product is then consistently wrong. Some such values break a rule and are refused, such as two colours with too little contrast or a resistor that puts a voltage outside its window, but whether a declaration is right is still decided by the person, the datasheet and the bench.

These rules change the role of an AI agent: it still edits declarations and code, but it is no longer trusted to keep anything consistent. Its mistakes are not prevented, but many are caught before they reach the product, with a message precise enough for the agent to correct itself. Hence the name: a fiducial is a fixed reference point, a universal measure that everything is aligned to and compared against, so it stays uniform. The declarations are that reference for a product.

IV. IMPLEMENTATION

Fiducial is a command-line tool, fid, built over six weeks by one person working with AI agents. Fig. 2 shows how a change moves through its parts.

A. One Structure for Every Product

fid new creates a product repository with a fixed layout, and fid add enables capabilities. A capability packages one discipline: its declarations, the generator that turns them into artifacts, its checks and instructions for agents. There are 23, from web applications and translations to firmware and hardware. Because every product has the same structure, whoever knows one knows them all, and a new product starts from parts that already work. fid upgrade brings platform improvements into each product while keeping its own edits.

B. One Typed Language Underneath

The platform, and the code products share through it, is written in Rust, a compiled language with a strict type system. The choice follows from the approach: every mistake the compiler refuses is one the checks never have to catch, and it is refused before anything runs, with a message naming the file and line – the signal an agent needs. Rust also needs no runtime or garbage collector, so one body of code can run wherever a product does. Its core is written without the standard library: the same message format, identifiers and typed physical quantities compile into a microcontroller's firmware, into the fid tool, and to WebAssembly, which runs them in the browser and on edge servers. The TypeScript types the web code uses are generated from the Rust ones, never written by hand, so the boundary between the two languages is not kept in step by hand either. TypeScript is used for interfaces, and Python only at build time, to drive the engines.

C. Declarations and Generation

Declarations are plain text files in version control, from the strings of each language to every part of a board with the net each pin connects to. Text was chosen over a graphical modelling tool because coding agents read and write it well, and every change then goes through ordinary code review. fid derive runs each capability's generator, which reads the declarations and writes the artifacts, from a typed module of translations to a firmware pin map or a KiCad circuit board [23]. A lock file records a cryptographic hash – a short fingerprint of a file's contents – for every input and output, so the platform knows exactly what each artifact was made from.

D. Checks

Checks run in continuous integration, the automatic build that runs on every proposed change, and a change is merged only when they pass. There are three kinds. Freshness: fid derive --check compares every recorded fingerprint with the repository and fails if a declaration changed without its artifacts being regenerated, or if a generated file was edited by hand. Rules: generation refuses declarations that break a rule, such as a language missing a string, two colours with too little contrast, parts that cost more than the declared limit, or a board that does not fit its case. Consumers: compilers and type checkers read the generated files, so code that still uses a renamed fact stops compiling.

E. Engines for Hard Problems

Some generation steps are real engineering problems: where to place the parts on a board, how to connect them, how to build a case around them. The first version placed parts with a hand-written search, and each new product added special rules until they began to interact. Now the platform only writes the problem down and hands it to an established engine. A constraint solver, CP-SAT [24], is given the rules a placement must satisfy and finds one that does, or proves none exists; build123d [12] builds the case; KiCad routes and checks the board. Every placement rule is one of eight general kinds, such as inside, apart or near, so a new product needs new declarations rather than new code.

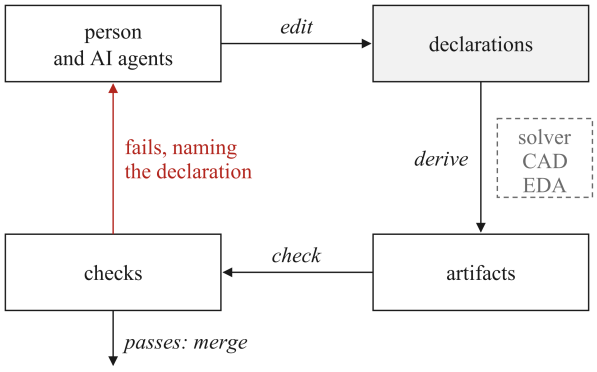


Fig. 2. How a change moves through Fiducial: declarations are edited, generation produces the artifacts with the help of established engines, and the checks either accept the change or return a message naming the declaration at fault.

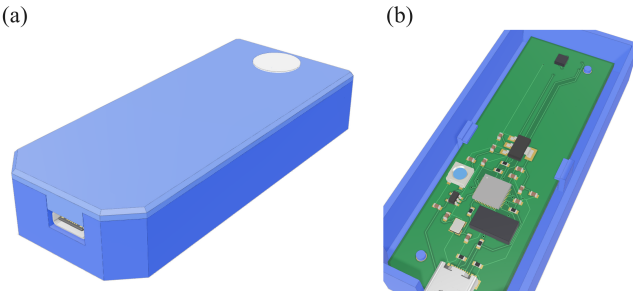


Fig. 3. The sensor stick, generated from its declaration: (a) the closed case; (b) the routed board inside it, with the sensor under its vent.

F. Agents in the Loop

Every product carries an AGENTS.md file and one instruction file per capability, so any agent starts from the same description of the product, and a hook stops it from editing a generated file. Agents fix a problem where the fact lives, never in the output. The platform grows the same way: a mistake that gets past the checks becomes a general rule for every product.

V. Example: A Sensor Stick

The repository contains a complete example: a USB-C stick that measures temperature and humidity and shows them on a web page (Fig. 3), with a printed case, a circuit board, Rust firmware and a browser page. The person writes the product declaration, an outline of its shape, the firmware's logic and the page; the routed board with its schematic and bill of materials, the case, the manufacturing files and the code both sides use to exchange readings are generated. Moving the sensor to another pin is one changed line: the check fails until the product is regenerated, after which the board, the firmware's pin map and the page's types have all followed with no other edit, and a pin the chip's sensor bus cannot use stops the firmware compiling.

VI. Evaluation

Checks are only useful if they catch the mistakes that people and agents actually make. The test borrows the idea of mutation testing [25]: a realistic mistake is introduced on purpose, the checks are run, and the result shows whether they noticed. Each mistake was applied to a clean copy of one of two products – a fresh web product in English and Serbian, and the sensor stick – the checks ran as in continuous integration, and the copy was reset. Four sets of 20 were used, each committed before its first run. The first covers eight kinds of drift met while building Fiducial and was used to improve it. The author wrote the second after the first was fixed; separate agents that read only the two products, not the platform, wrote the third and the fourth, the fourth aimed at the classes the third revealed. As a baseline, every set also ran through ordinary CI alone – the same compilers, type checkers and tests, with Fiducial's two steps removed. Three correct changes and two runs with no change tested the opposite direction: that the checks do not fail without a reason. Table II gives the results.

TABLE II. INJECTED MISTAKES CAUGHT

Mistakes	Ordinary CI	First run	Fiducial 0.10.1
First, used to improve	1/20	15/20	20/20
Second, by the author	0/20	15/20	20/20
Third, by an agent	0/20	16/20	20/20
Fourth, by an agent	0/20	14/20	19/20
Correct changes	5/5	5/5	5/5

On first run the checks caught 15, 16 and 14 of the three unseen sets; ordinary CI caught none of them, and one of the first set, because nothing in it compares one artifact with another. Every miss fell into one of five classes: a declared value typed again into code, such as a footer spelling out the domain; the check itself removed, or run after a step that regenerates what it checks; a field that accepts any value of the right shape, such as a month that does not exist; a rule from outside the product, such as a regulator whose output is not the rail it feeds or a linker script larger than the flash chip; and a generator reading a file nobody declared.

Fixing misses one rule at a time had taken the second set to 20 and left the third at 16, so each class now has one mechanism: a scan for declared values in hand-written code; a check that CI still runs the gate first; typed declarations that reject unknown keys; outside rules kept as data, from datasheets, fabrication limits and law; and generators confined to their declared inputs. With these, release 0.10.1 catches every mistake in the first three sets and 19 of the fourth, and every correct change still passes. The one it misses is meaning: a colour pair relabelled as large text passes, because which text is large is declared nowhere.

All four sets and their script are in the repository. Only first runs measure how the checks generalise; the fourth set was aimed at classes the author named; and two products have been built, neither yet manufactured. The test measures the checks, not whether agents make fewer mistakes with Fiducial.

VII. CONCLUSION

AI agents have made writing each part of a product fast, and that has moved the hard problem to keeping the parts in agreement. Better models will move the line between what a person checks and what is left to agents, but a probabilistic system cannot guarantee its output, and even a deterministic tool is only right about what it was told. Fiducial answers this at the level of the product rather than the codebase: it keeps the parts consistent with each other, while whether each declared fact is true remains the person's to judge. On 60 mistakes it had not seen, its checks caught 45 on first run where ordinary CI caught none, each with a message that names the cause, and every miss fell into one of five classes that are now each closed by one mechanism; what remains is meaning, which no declaration states. The next steps are to test it on mistakes written by other people; to explore retrieval-augmented generation, so that agents are given the declarations and documentation a task needs rather than whole repositories; to refine the platform by applying it to new products, the first a school building of several controllers and a server; and to measure and lower the tokens agents spend, reasoning tokens above all. Fiducial and its example are open source [4].

REFERENCES

[1] Claude Code documentation. Available at: https://docs.anthropic.com/en/docs/claude-code

[2] OpenAI Codex. Available at: https://openai.com/codex/

[3] A. Zdravković, Đ. Jovanović and P. Šubarević, "Outreach: Modular AI-enhanced off-grid mesh communication system for remote environments," IEEESTEC – 18th Student Projects Conference, Niš, 2025.

[4] A. Zdravković, Fiducial, release 0.10.1, 2026. Available at: https://github.com/AleksaZCodes/fiducial

[5] D. C. Schmidt, "Guest editor's introduction: Model-driven engineering," Computer, vol. 39, no. 2, pp. 25–31, 2006.

[6] M. Brambilla, J. Cabot and M. Wimmer, Model-Driven Software Engineering in Practice, 2nd ed. Morgan & Claypool, 2017.

[7] S. Friedenthal, A. Moore and R. Steiner, A Practical Guide to SysML, 3rd ed. Morgan Kaufmann, 2014.

[8] B. Nuseibeh, S. Easterbrook and A. Russo, "Leveraging inconsistency in software development," Computer, vol. 33, no. 4, pp. 24–29, 2000.

[9] S. J. I. Herzig, A. Qamar and C. J. J. Paredis, "An approach to identifying inconsistencies in model-based systems engineering," Procedia Computer Science, vol. 28, pp. 354–362, 2014.

[10] Altium, MCAD CoDesigner, ECAD–MCAD collaboration. Available at: https://www.altium.com/

[11] atopile, a language for designing circuit boards as code. Available at: https://github.com/atopile/atopile

[12] build123d, a Python CAD library on the OpenCascade kernel. Available at: https://github.com/gumyr/build123d

[13] Protocol Buffers. Available at: https://protobuf.dev/

[14] Bazel, a build and test tool. Available at: https://bazel.build/

[15] E. Dolstra, M. de Jonge and E. Visser, "Nix: A safe and policy-free system for software deployment," in Proc. LISA, 2004, pp. 79–92.

[16] AGENTS.md, a format for guiding coding agents. Available at: https://agents.md/

[17] M. Chen et al., "Evaluating large language models trained on code," arXiv:2107.03374, 2021.

[18] J. Liu, C. S. Xia, Y. Wang and L. Zhang, "Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation," in Proc. NeurIPS, 2023.

[19] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt and R. Karri, "Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions," in Proc. IEEE S&P, 2022, pp. 754–768.

[20] C. E. Jimenez et al., "SWE-bench: Can language models resolve real-world GitHub issues?," in Proc. ICLR, 2024.

[21] J. Huang et al., "Large language models cannot self-correct reasoning yet," in Proc. ICLR, 2024.

[22] S. Kambhampati et al., "Position: LLMs can't plan, but can help planning in LLM-Modulo frameworks," in Proc. ICML, PMLR 235, 2024.

[23] KiCad EDA. Available at: https://www.kicad.org/

[24] L. Perron and F. Didier, CP-SAT, Google OR-Tools. Available at: https://developers.google.com/optimization/cp/cp_solver

[25] R. A. DeMillo, R. J. Lipton and F. G. Sayward, "Hints on test data selection: Help for the practicing programmer," Computer, vol. 11, no. 4, pp. 34–41, 1978.